

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 3

Module 7: Lists

Module 8: Tuples

Module 9: Sets

Module 10: Dictionaries

Continues from Part 1-2 (Modules 1-6)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
MODULE 7: LISTS	4
7.1 Creating Lists	4
7.2 Indexing	4
7.3 Slicing	4
7.4 List Methods	4
7.5 List Comprehension	5
7.6 Nested Lists	5
Real-Life Example: Student Marks List	5
Programs — Module 7	6
Interview Questions — Module 7	6
MCQs — Module 7	7
Assignments — Module 7	7
MODULE 8: TUPLES	8
8.1 Creation	8
8.2 Operations	8
8.3 Methods	8
8.4 Packing and Unpacking	8
Programs — Module 8	9
Interview Questions — Module 8	9
MCQs — Module 8	9
Exercises — Module 8	10
MODULE 9: SETS	11
9.1 Creating Sets	11
9.2 Set Operations	11
9.3 Set Methods	11
Real-Life Example: Removing Duplicate Entries	12
Interview Questions — Module 9	12
MCQs — Module 9	12
Exercises — Module 9	13
MODULE 10: DICTIONARIES	14
10.1 Key-Value Pairs	14
10.2 Dictionary Methods	14
10.3 Nested Dictionaries	14
10.4 Applications	15

Programs — Module 10	15
Interview Questions — Module 10	15
MCQs — Module 10	16
Exercises — Module 10	16
QUICK REVISION NOTES — MODULES 7-10	17
LIST vs TUPLE vs SET vs DICTIONARY	18
COMMON ERRORS AND FIXES — MODULES 7-10	19
CODING BEST PRACTICES — MODULES 7-10	20

MODULE 7: LISTS

A list is an ordered, changeable (mutable) collection that can hold items of different data types together. Lists are one of the most commonly used data structures in Python — think of them as a container you can keep adding to, removing from, and reordering.

7.1 Creating Lists

```
fruits = ["apple", "banana", "cherry"]
mixed = [1, "hello", 3.5, True]
empty = []
numbers = list(range(1, 6))  # [1, 2, 3, 4, 5]
```

Real-World Analogy

A list is like a shopping trolley — you can add items, remove items, and reorder them at any time, and it can hold different kinds of things together (vegetables, boxes, bottles).

7.2 Indexing

```
fruits = ["apple", "banana", "cherry"]
print(fruits[0])    # first item
print(fruits[-1])   # last item
```

Output:

```
apple
cherry
```

7.3 Slicing

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4])    # index 1 to 3
print(nums[:3])     # first three
print(nums[::-1])   # reversed list
```

Output:

```
[20, 30, 40]
[10, 20, 30]
[50, 40, 30, 20, 10]
```

7.4 List Methods

Method	Purpose	Example	Result
append()	Adds an item to the end	<code>l.append(4)</code>	<code>[1,2,3,4]</code>
insert()	Inserts item at a given index	<code>l.insert(1,99)</code>	<code>[1,99,2,3]</code>
remove()	Removes the first matching value	<code>l.remove(2)</code>	<code>[1,3]</code>

Method	Purpose	Example	Result
pop()	Removes & returns item at index (default last)	<code>l.pop()</code>	removes last item
sort()	Sorts the list in place (ascending)	<code>l.sort()</code>	[1,2,3]
reverse()	Reverses the list in place	<code>l.reverse()</code>	[3,2,1]
index()	Returns index of first match	<code>l.index(2)</code>	1
count()	Counts occurrences of a value	<code>l.count(2)</code>	1
extend()	Appends all items from another list	<code>l.extend([4,5])</code>	[1,2,3,4,5]
clear()	Removes all items	<code>l.clear()</code>	[]

7.5 List Comprehension

A compact, Pythonic way to build a new list from an existing sequence in a single line, optionally with a filter condition.

```
squares = [x**2 for x in range(1, 6)]
print(squares)

evens = [x for x in range(1, 11) if x % 2 == 0]
print(evens)
```

Output:

```
[1, 4, 9, 16, 25]
[2, 4, 6, 8, 10]
```

This is equivalent to (but shorter than) the traditional loop version:

```
squares = []
for x in range(1, 6):
    squares.append(x**2)
```

7.6 Nested Lists

A list that contains other lists as its elements — commonly used to represent tables, grids, or matrices.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(matrix[1])      # second row
print(matrix[1][2])   # row 1, column 2
```

Output:

```
[4, 5, 6]
6
```

Real-Life Example: Student Marks List

```
students = [
    ["Riya", 88],
    ["Aman", 72],
    ["Neha", 95],
]
for name, marks in students:
    print(f"{name}: {marks}")
```

Output:

```
Riya: 88
Aman: 72
Neha: 95
```

Programs — Module 7

Program: Find the Largest Number in a List (without max())

```
nums = [23, 67, 12, 89, 45]
largest = nums[0]
for n in nums:
    if n > largest:
        largest = n
print("Largest:", largest)
```

Line	Explanation
2	Assume the first item is the largest so far.
3-5	Compare every item; whenever a bigger one is found, update largest.
6	After the loop finishes, largest holds the maximum value.

Output:

```
Largest: 89
```

Interview Questions — Module 7**Q1. What is the difference between a list and a tuple?**

Ans: A list is mutable (can be changed after creation — items added, removed, modified) and uses square brackets []. A tuple is immutable (cannot be changed once created) and uses round brackets ().

Q2. What is list comprehension and why use it?

Ans: A concise one-line syntax for creating a new list from an iterable, optionally applying a condition or transformation. It is more readable and often faster than an equivalent for loop with append().

Q3. What is the difference between append() and extend()?

Ans: append() adds its argument as a single new item at the end (even if that argument is a list). extend() adds each element of the given iterable individually to the end.

Q4. What happens when you call pop() on an empty list?

Ans: It raises an `IndexError: pop from empty list`, since there is no element to remove.

Q5. How do you create a copy of a list without affecting the original?

Ans: Using `list.copy()`, the slice `l[:]`, or `list(l)` — a plain `l2 = l1` only copies the reference, so changes to `l2` would also affect `l1`.

MCQs — Module 7

No.	Question	Answer	Options (A-D)
1	Lists in Python are:	A	A) Mutable B) Immutable C) Fixed-size D) Read-only
2	Which method adds one item to the end of a list?	B	A) add() B) append() C) insert() D) push()
3	<code>[x for x in range(3)]</code> produces:	C	A) (0,1,2) B) {0,1,2} C) [0,1,2] D) Error
4	Which method removes and returns the last item?	D	A) remove() B) delete() C) del D) pop()
5	<code>l = [1,2,3]; l.extend([4,5])</code> results in:	A	A) [1,2,3,4,5] B) [1,2,3,[4,5]] C) Error D) [4,5]
6	Negative indexing in a list starts from:	B	A) 0 B) -1 C) -0 D) 1
7	<code>matrix[1][2]</code> refers to:	C	A) Row 2, Col 1 B) Whole row 1 C) Row 1, Col 2 D) Error

Assignments — Module 7

- Write a program to remove duplicate elements from a list.
- Write a program to find the second largest number in a list.
- Write a program to merge two lists and sort the result.
- Write a program using list comprehension to create a list of squares of odd numbers from 1-20.
- Write a program to transpose a 3x3 matrix represented as a nested list.
- Write a program to count how many times each element appears in a list (without using Counter).

MODULE 8: TUPLES

A tuple is an ordered collection like a list, but immutable — once created, its contents cannot be changed. Tuples use round brackets ().

8.1 Creation

```
coordinates = (10, 20)
single = (5,)      # comma required for a one-item tuple
mixed = (1, "hi", 3.5)
no_brackets = 1, 2, 3  # brackets are optional
```

Common Beginner Mistake

Writing `x = (5)` does NOT create a tuple — it is just the integer 5 in parentheses. A single-item tuple needs a trailing comma: `x = (5,)`.

8.2 Operations

```
t = (1, 2, 3)
print(t + (4, 5))  # concatenation → new tuple
print(t * 2)       # repetition
print(2 in t)      # membership
print(len(t))      # length
```

Output:

```
(1, 2, 3, 4, 5)
(1, 2, 3, 1, 2, 3)
True
3
```

8.3 Methods

Because tuples are immutable, they support only two methods:

Method	Purpose	Example	Result
<code>count()</code>	Counts occurrences of a value	<code>(1,2,2).count(2)</code>	2
<code>index()</code>	Returns index of first match	<code>(1,2,3).index(3)</code>	2

8.4 Packing and Unpacking

Packing: combining multiple values into a tuple automatically. Unpacking: extracting tuple values into separate variables in one line.


```

point = 10, 20, 30      # packing
x, y, z = point         # unpacking
print(x, y, z)

a, *rest = (1, 2, 3, 4) # star unpacking – 'rest' collects extras
print(a, rest)

```

Output:

```

10 20 30
1 [2, 3, 4]

```

Programs — Module 8

Program: Swap Two Variables Using Tuple Packing/Unpacking

```

a, b = 5, 10
a, b = b, a
print("a =", a, "b =", b)

```

Output:

```

a = 10 b = 5

```

Interview Questions — Module 8**Q1. Why use a tuple instead of a list?**

Ans: Tuples are immutable, so they are useful for fixed data that should not change (e.g., coordinates, RGB colour values). They are also slightly faster and can be used as dictionary keys, unlike lists.

Q2. Can a tuple contain a mutable object like a list?

Ans: Yes. The tuple itself cannot be reassigned or resized, but if one of its elements is a mutable object (like a list), that inner object can still be modified in place.

Q3. How do you create a tuple with a single element?

Ans: Add a trailing comma: `t = (5,)`. Without the comma, `(5)` is just an integer, not a tuple.

Q4. What is tuple unpacking?

Ans: Assigning the elements of a tuple to multiple variables in a single statement, e.g., `x, y = (1, 2)`, matching by position.

Q5. Can tuples be used as dictionary keys?

Ans: Yes, because tuples are immutable and hashable (as long as all their elements are also hashable), unlike lists which cannot be used as dictionary keys.

MCQs — Module 8

No.	Question	Answer	Options (A-D)
1	Tuples are:	B	A) Mutable B) Immutable C) Unordered D) Key-value based
2	Which creates a single-element tuple?	C	A) (5) B) [5] C) (5,) D) {5}
3	<code>t = (1,2,3); t[0] = 9</code> results in:	D	A) (9,2,3) B) [9,2,3] C) No change D) TypeError

No.	Question	Answer	Options (A-D)
4	How many built-in methods do tuples support?	A	A) 2 B) 5 C) 10 D) 0
5	a, *b = (1,2,3,4) makes b equal to:	C	A) 4 B) (2,3,4) C) [2,3,4] D) Error

Exercises — Module 8

1. Write a program to find the maximum and minimum in a tuple without using max()/min().
2. Write a program that unpacks a tuple of 5 subjects into named variables and prints them.
3. Write a program to convert a list into a tuple and vice versa.
4. Write a program to count how many times a value appears in a tuple using count().

MODULE 9: SETS

A set is an unordered collection of unique items — duplicates are automatically removed. Sets are written with curly braces `{}` and are ideal whenever you need to eliminate duplicates or perform mathematical set operations.

9.1 Creating Sets

```
fruits = {"apple", "banana", "cherry"}
nums = set([1, 2, 2, 3, 3, 3]) # duplicates removed automatically
print(nums)
empty = set() # {} alone creates an empty DICT, not a set!
```

Output:

```
{1, 2, 3}
```

Common Beginner Mistake

`x = {}` creates an empty dictionary, not an empty set — Python reserves `{}` for dict. Use `x = set()` to create an empty set.

9.2 Set Operations

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a | b) # union — all unique items from both
print(a & b) # intersection — common items
print(a - b) # difference — in a but not in b
print(a ^ b) # symmetric difference — in either, not both
```

Output:

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

Venn-diagram style summary:

Set A	Set B
(1,2,3,4)	(3,4,5,6)
[1,2	[3,4] 5,6]
A-only	AnB B-only

```
Union (A|B)      = everything: 1,2,3,4,5,6
Intersection(A&B)= overlap only: 3,4
Difference (A-B) = A-only: 1,2
Sym Diff (A^B)   = A-only + B-only: 1,2,5,6
```

9.3 Set Methods

Method	Purpose	Example	Result
--------	---------	---------	--------

Method	Purpose	Example	Result
add()	Adds one item	s.add(5)	{...,5}
remove()	Removes item; error if missing	s.remove(2)	raises error if absent
discard()	Removes item; no error if missing	s.discard(2)	safe removal
union()	Same as operator	a.union(b)	combined set
intersection()	Same as & operator	a.intersection(b)	common items
clear()	Empties the set	s.clear()	set()

Real-Life Example: Removing Duplicate Entries

```
emails = ["a@x.com", "b@x.com", "a@x.com", "c@x.com"]
unique_emails = set(emails)
print(unique_emails)
```

Output:

```
{'a@x.com', 'b@x.com', 'c@x.com'}
```

Interview Questions — Module 9

Q1. What is the key property that makes sets useful?

Ans: Sets automatically store only unique elements and have no defined order, making them ideal for removing duplicates and performing fast membership tests and mathematical set operations.

Q2. What is the difference between `remove()` and `discard()`?

Ans: `remove()` raises a `KeyError` if the item is not found in the set. `discard()` does the same job but silently does nothing if the item is missing, making it the safer choice when you're unsure the item exists.

Q3. Can a set contain duplicate values?

Ans: No — a set automatically eliminates duplicates; adding an already-present value has no effect.

Q4. Why can't a set contain a list as an element?

Ans: Set elements must be hashable (immutable), and lists are mutable, so they cannot be hashed or stored inside a set. Tuples, being immutable, can be stored instead.

Q5. How do you find common elements between two sets?

Ans: Using the `intersection()` method or the `&` operator, e.g., `set_a & set_b`.

MCQs — Module 9

No.	Question	Answer	Options (A-D)
1	Sets are:	C	A) Ordered, duplicate-allowed B) Indexed C) Unordered, unique items only D) Key-value pairs
2	<code>x = {}</code> creates:	B	A) Empty set B) Empty dictionary C) Error D) Empty list
3	Which operator gives the union of two sets?	A	A) B) & C) - D) ^

No.	Question	Answer	Options (A-D)
4	Which method removes an item without raising an error if absent?	D	A) remove() B) pop() C) delete() D) discard()
5	{1,2,3} & {2,3,4} equals:	B	A) {1,2,3,4} B) {2,3} C) {1,4} D) {}

Exercises — Module 9

1. Write a program to remove duplicate elements from a list using a set.
2. Write a program to find common elements between two lists using set intersection.
3. Write a program to check if one set is a subset of another.
4. Write a program to find elements present in exactly one of two sets (symmetric difference).

MODULE 10: DICTIONARIES

A dictionary stores data as key-value pairs, where each unique key maps to a value — similar to a real dictionary where a word (key) maps to its meaning (value). Dictionaries use curly braces { } with key:value syntax.

10.1 Key-Value Pairs

```
student = {"name": "Riya", "age": 21, "course": "B.Tech"}
print(student["name"])      # access by key
print(student.get("age"))   # safer access
student["age"] = 22         # update a value
student["grade"] = "A"      # add a new key-value pair
print(student)
```

Output:

```
Riya
21
{'name': 'Riya', 'age': 22, 'course': 'B.Tech', 'grade': 'A'}
```

dict[key] vs .get(key)

student['x'] raises a KeyError if 'x' does not exist. student.get('x') returns None (or a default you specify) instead of crashing — safer when the key might be missing.

10.2 Dictionary Methods

Method	Purpose	Example	Result
keys()	Returns all keys	d.keys()	dict_keys([...])
values()	Returns all values	d.values()	dict_values([...])
items()	Returns key-value pairs	d.items()	dict_items([(k,v),...])
get()	Safely retrieves a value	d.get("x")	value or None
update()	Merges another dict in	d.update({"y":1})	adds/updates keys
pop()	Removes key & returns its value	d.pop("x")	value removed
setdefault()	Gets value, or sets it if missing	d.setdefault("z",0)	0 if absent

10.3 Nested Dictionaries

A dictionary can store other dictionaries as values — useful for representing structured records, like a database of multiple students.

```
students = {
    "s1": {"name": "Riya", "marks": 88},
    "s2": {"name": "Aman", "marks": 72},
}
print(students["s1"]["name"])
for sid, info in students.items():
    print(sid, "->", info["name"], info["marks"])
```

Output:

```
Riya
s1 -> Riya 88
s2 -> Aman 72
```

10.4 Applications

- Storing structured records — a user profile, a product catalogue entry.
- Counting frequency of items (e.g., word count in a paragraph).
- Representing JSON data from APIs — JSON objects map naturally onto Python dictionaries.
- Fast lookups — checking membership or retrieving a value by key is much faster than searching a list.

Programs — Module 10

Program: Word Frequency Counter

```
text = "the cat sat on the mat the cat ran"
words = text.split()
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

Line	Explanation
2	Split the sentence into a list of words.
3	Start with an empty dictionary to hold counts.
4-5	For each word, <code>freq.get(word, 0)</code> returns the current count (or 0 if new), then adds 1 and stores it back.

Output:

```
{'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}
```

Interview Questions — Module 10

Q1. What is the difference between `dict[key]` and `dict.get(key)`?

Ans: `dict[key]` raises a `KeyError` if the key does not exist. `dict.get(key)` returns `None` (or a specified default) instead, avoiding a crash — safer when a key might be missing.

Q2. Can dictionary keys be any data type?

Ans: Keys must be immutable/hashable types — strings, numbers, or tuples (of immutable items). Lists and other dictionaries cannot be used as keys because they are mutable.

Q3. How do you iterate over both keys and values of a dictionary?

Ans: Using the `items()` method: `for key, value in d.items():` — this unpacks each key-value pair directly.

Q4. Are Python dictionaries ordered?

Ans: As of Python 3.7+, dictionaries maintain insertion order as a guaranteed language feature (earlier versions did not guarantee this).

Q5. What is the time complexity of looking up a value by key in a dictionary?

Ans: $O(1)$ on average — dictionaries are implemented using hash tables, so lookups, insertions, and deletions are typically constant time.

Q6. What happens if you assign to an existing key?

Ans: The old value is overwritten with the new one; dictionary keys are always unique, so there can never be two entries with the same key.

MCQs — Module 10

No.	Question	Answer	Options (A-D)
1	Dictionaries store data as:	B	A) Ordered index pairs B) Key-value pairs C) Unique values only D) Tuples only
2	Which method safely retrieves a value without raising an error?	C	A) <code>fetch()</code> B) <code>find()</code> C) <code>get()</code> D) <code>search()</code>
3	<code>d.items()</code> returns:	D	A) Only keys B) Only values C) A list D) Key-value pairs
4	Since which Python version do dicts guarantee insertion order?	A	A) 3.7 B) 2.7 C) 3.0 D) 3.10
5	Which of these CANNOT be a dictionary key?	B	A) String B) List C) Tuple D) Integer
6	Average time complexity of a dictionary lookup:	A	A) $O(1)$ B) $O(n)$ C) $O(\log n)$ D) $O(n^2)$

Exercises — Module 10

- Write a program to merge two dictionaries.
- Write a program to find the key with the maximum value in a dictionary.
- Write a program that counts the frequency of each character in a string using a dictionary.
- Write a program to invert a dictionary (swap its keys and values).
- Create a nested dictionary of 3 employees (name, salary, department) and print each employee's details.

QUICK REVISION NOTES — MODULES 7-10

- List `[]`: ordered, mutable, allows duplicates. Best for general-purpose collections you'll modify.
- Tuple `()`: ordered, immutable, allows duplicates. Best for fixed data and dictionary keys.
- Set `{}`: unordered, mutable, NO duplicates. Best for uniqueness checks and set math (union/intersection).
- Dict `{key:value}`: unordered-by-concept but insertion-ordered since 3.7, mutable, unique keys. Best for fast lookups by name/ID.
- List comprehension `[expr for item in seq if cond]` is the Pythonic way to build lists in one line.
- `x = {}` makes an empty DICT, not a set — use `set()` for an empty set.
- `dict.get(key)` and `set.discard(item)` are the 'safe' versions that avoid errors when something might be missing.

LIST vs TUPLE vs SET vs DICTIONARY

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ }	{k:v}
Ordered	Yes	Yes	No	Yes (3.7+)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Access by	Index	Index	Value (no index)	Key

COMMON ERRORS AND FIXES — MODULES 7-10

Error	Cause	Fix
IndexError: list index out of range	Accessing an index beyond the list length	Check len(list) before indexing
TypeError: 'tuple' object does not support item assignment	Trying to modify a tuple	Tuples are immutable — convert to list first if changes are needed
TypeError: unhashable type: 'list'	Using a list as a set element or dict key	Use a tuple instead of a list
KeyError	Accessing a dictionary key that doesn't exist	Use dict.get(key) or check 'key in dict' first
x = {} treated as empty set by mistake	{ } is actually an empty dict	Use set() to create an empty set

CODING BEST PRACTICES — MODULES 7-10

- Choose the right structure: list for ordered/changeable data, tuple for fixed data, set for uniqueness, dict for key-based lookup.
- Prefer list comprehensions over manual `append()` loops for simple transformations — they're more readable.
- Use `dict.get()` instead of `dict[key]` when a key might not exist, to avoid crashing the program.
- Avoid modifying a list while iterating over it directly — iterate over a copy (`for x in list[:]`) if you need to remove items.
- Use meaningful key names in dictionaries (e.g., `'student_name'` not `'sn'`) for readability.